Software Testing and Quality Assurance

Assignment-4

Q1. What is a Defect? Explain the importance of defect tracking in software testing.

A Defect is an error, flaw, or mistake in software that causes the application to behave differently from its expected results or specified requirements. A defect may occur due to incorrect coding, incomplete requirements, design mistakes, or errors during software development.

For example, if a banking application deducts money from an account but does not update the account balance correctly, it is considered a software defect.

Importance of Defect Tracking in Software Testing:

1. Early Identification of Errors: Defect tracking helps testers identify and record errors as soon as they are discovered. Early detection reduces the cost and effort required to fix defects.

2. Improves Software Quality: By recording and resolving defects systematically, the testing team ensures that the application works according to the specified requirements.

3. Maintains Accountability: Defects can be assigned to specific developers or team members, making it easier to track responsibility and monitor progress.

4. Provides Clear Communication: Defect tracking creates a common record of issues for testers, developers, project managers, and other stakeholders.

5. Helps Prioritize Defects: Defects can be classified according to severity and priority so that critical problems are resolved before minor issues.

6. Supports Progress Monitoring: The team can monitor how many defects are open, fixed, retested, or closed. This helps assess the overall progress of testing.

7. Prevents Repeated Errors: Maintaining a record of previous defects helps teams understand recurring problems and improve development practices.

Q2. Explain the Defect Life Cycle with a suitable diagram.

The Defect Life Cycle is the sequence of stages through which a defect passes from the time it is identified until it is resolved and closed. It helps the testing and development teams manage defects systematically and monitor their progress.

A defect may move through different states depending on its status, the actions taken by the team, and the results of retesting.

Explanation of the stages:

1. New: The tester identifies a defect and records it in the defect tracking system with the necessary details.
2. Assigned: The defect is assigned to the responsible developer or team for investigation and correction.
3. Open / In Progress: The developer examines the defect, identifies its cause, and begins working on a solution.
4. Fixed: The developer makes the necessary code changes and marks the defect as fixed.
5. Retest: The tester tests the corrected functionality to verify whether the defect has been resolved.
6. Closed: If the retest is successful, the tester closes the defect.
7. Reopened: If the defect still exists or occurs again, the tester reopens it and sends it back for further investigation.

Other possible states include Rejected, Duplicate, Deferred, and Not Reproducible.

The defect life cycle ensures that every reported defect is tracked, investigated, corrected, and verified properly.

Q3. What are the different states of a defect in the Defect Life Cycle? Explain each state.

A defect state represents the current status of a defect during its life cycle. These states help the testing team understand which defects require action and which have been resolved.

Different states of a defect:

1. New: This is the initial state in which a tester reports a newly discovered defect. The defect is recorded with a description, reproduction steps, expected result, and actual result.

2. Assigned: The defect is assigned to a developer or responsible team member who will investigate and fix it.

3. Open: The defect has been accepted for investigation and requires further analysis or correction. Some projects use Open to indicate that work has not yet started, while others use In Progress for active work.

4. In Progress: The developer is actively investigating the cause of the defect and implementing a solution.

5. Fixed: The developer has made a correction and updated the defect status. However, the fix must still be verified by the testing team.

6. Retest: The tester checks the corrected functionality to confirm that the defect has been resolved.

7. Verified: The tester confirms that the defect no longer occurs and that the correction produces the expected result. Some tools combine verification with retesting.

8. Closed: The defect is considered resolved after successful verification. No further action is required unless the problem appears again.

9. Reopened: If the defect persists after the fix or reappears during testing, the tester reopens it for additional correction.

10. Rejected / Invalid: The reported issue is rejected because it is not a genuine defect, the behaviour matches the requirements, or the report is otherwise invalid.

Q4. Explain the process of Defect Tracking and Reporting in software testing.

Defect Tracking and Reporting is the process of recording, monitoring, communicating, and managing defects discovered during software testing. It ensures that every identified issue is documented, assigned to the appropriate person, corrected, and verified.

Process of Defect Tracking and Reporting:

1. Defect Identification: The tester executes test cases and compares actual results with expected results. If a mismatch is found, the tester identifies it as a possible defect.

2. Defect Logging: The tester records the defect in a tracking tool or defect report. The report includes the defect title, description, reproduction steps, expected result, actual result, and supporting evidence.

3. Defect Classification: The defect is classified according to its type, such as functional, performance, security, usability, or compatibility. Its severity and priority are also determined.

4. Defect Assignment: The defect is assigned to the appropriate developer or team based on the affected module and the expertise required to resolve it.

5. Defect Investigation: The developer examines the report, reproduces the issue, identifies its root cause, and determines the necessary correction.

6. Defect Fixing: The developer modifies the code or configuration to correct the problem and updates the defect status.

7. Retesting: The tester executes the relevant test cases again to check whether the defect has been corrected successfully.

8. Regression Testing: Related features and existing functionality are tested to ensure that the correction has not introduced new defects.

9. Defect Closure or Reopening: If the correction works as expected, the tester closes the defect. If the issue remains, the defect is reopened for further investigation.

10. Defect Reporting and Monitoring: The testing team prepares reports showing the total number of defects, open and closed defects, severity distribution, and resolution progress. These reports help managers evaluate software quality and testing progress.

Q5. What information should be included in a Defect Report? Explain.

A Defect Report is a document or record that provides complete information about an error found during software testing. It helps developers understand, reproduce, investigate, and correct the problem.

A clear and accurate defect report reduces communication gaps and helps the testing team verify the correction efficiently.

Important information included in a Defect Report:

1. Defect ID: A unique identification number assigned to each defect so that it can be tracked easily throughout its life cycle.

2. Defect Title: A short and meaningful summary describing the problem. For example, "Login fails with valid credentials."

3. Defect Description: A detailed explanation of the issue, including the circumstances in which it occurs and its effect on the application.

4. Steps to Reproduce: A numbered list of actions that allows the developer to reproduce the defect consistently.

5. Expected Result: The behaviour or output that should occur according to the software requirements.

6. Actual Result: The behaviour or output observed during test execution. Comparing it with the expected result helps explain the defect.

7. Severity: Indicates the technical impact of the defect on the application. For example, a complete application crash may be classified as critical or blocker severity.

8. Priority: Indicates how urgently the defect should be fixed. A defect affecting a major business function may receive high priority.

9. Environment Details: Includes relevant information such as operating system, browser, device, application version, database, and network configuration.

10. Reported By and Reported Date: Identifies the person who reported the defect and the date on which it was discovered.

Q6. Differentiate between Defect Severity and Defect Priority with suitable examples.

Defect Severity refers to the degree of impact a defect has on the software's functionality or operation. It indicates how seriously the defect affects the system.

Defect Priority refers to the urgency with which a defect should be fixed. It helps the project team decide which defects need attention first.

Although severity and priority are related, they represent different concepts. Severity mainly concerns the technical impact, while priority concerns the urgency of correction.

Basis	**Defect Severity**	**Defect Priority**
Meaning	Measures the impact of a defect on software functionality.	Measures the urgency of fixing a defect.
Focus	How seriously the defect affects the system.	How soon the defect should be corrected.
Usually determined by	Testers, QA engineers, and technical teams.	Product owners, project managers, QA leads, and other stakeholders.

Main factors	Functional impact, system failure, data loss, and technical consequences.	Business needs, deadlines, customer impact, and release plans.
Common levels	Critical, Major, Moderate, Minor.	High, Medium, Low.
Effect on workflow	Helps assess the seriousness of the problem.	Helps determine the order of fixing defects.
Relationship	A severe defect may have high or low priority depending on context.	A high-priority defect may be minor in technical severity.

Examples of Defect Severity and Priority:

1. High Severity, High Priority: The payment system fails for all customers during an active online sale. It prevents essential transactions and requires immediate correction.

2. High Severity, Low Priority: A serious failure occurs in a rarely used legacy feature that is scheduled for removal in the next release. Its technical impact is high, but its immediate business urgency may be lower.

3. Low Severity, High Priority: The company's name is misspelled on the main page just before a major product launch. The technical impact is minor, but the issue may require urgent correction because of its visibility.

4. Low Severity, Low Priority: A small alignment issue appears on an infrequently used settings page. It does not affect functionality and can be fixed later.

Conclusion: Severity helps the team understand the seriousness of a defect, whereas priority helps decide when it should be fixed. Considering both allows the team to allocate resources effectively and address the most important problems first.

Q7. Explain the Defect Management Process in detail.

Defect Management is a systematic process of identifying, recording, classifying, prioritizing, assigning, resolving, tracking, and closing defects found during software development and testing. Its purpose is to ensure that defects are handled effectively and that software quality continuously improves.

Defect management involves coordination among testers, developers, project managers, and other stakeholders. It also provides information about the stability of the software and the effectiveness of testing.

Steps involved in the Defect Management Process:

1. Defect Prevention: The process begins by identifying common causes of defects and taking steps to prevent them. Activities such as requirement reviews, design inspections, coding standards, and developer training can reduce the number of defects introduced into the software.

2. Defect Identification: Testers, developers, or users identify defects through test execution, code reviews, inspections, monitoring, or feedback. Each suspected defect is examined to determine whether the observed behaviour is incorrect.

3. Defect Logging: Every confirmed or suspected defect is recorded in a defect tracking system. The report includes the description, steps to reproduce, expected and actual results, environment, and relevant evidence.

4. Defect Classification: Defects are categorized according to their nature, such as functional, performance, security, usability, or compatibility defects. This classification helps the team understand the type of problems occurring in the software.

5. Defect Severity and Priority Assessment: The team evaluates the impact of each defect and determines how urgently it should be fixed. Critical defects affecting major functions or data integrity generally receive immediate attention.

6. Defect Assignment: The defect is assigned to a suitable developer or team based on the affected module, technical expertise, workload, and project responsibilities.

7. Defect Investigation and Resolution: The assigned developer investigates the problem, identifies its root cause, and implements an appropriate correction. The solution and relevant details are recorded in the tracking tool.

8. Retesting and Verification: After the developer marks the defect as fixed, the tester executes the relevant test cases again. If the expected result is achieved, the correction is verified.

9. Regression Testing: The testing team checks related features and previously working functionality to ensure that the new correction has not caused additional problems.

10. Defect Closure or Reopening: A verified defect is closed. If the problem persists or reappears, it is reopened and returned for further investigation and correction.

Q8. What are the main activities involved in Defect Management? Explain each activity.

Defect management consists of several connected activities that help the testing team handle software defects from identification to final resolution. Each activity has a specific purpose and contributes to the quality and reliability of the software.

Main activities involved in defect management:

1. Defect Prevention: This activity focuses on reducing the possibility of defects occurring in the first place. Requirement analysis, design reviews, coding standards, and developer training help prevent common errors.

2. Defect Discovery: Defects are discovered during software testing, code reviews, inspections, integration, system execution, or user feedback. Testers compare actual behaviour with expected behaviour to identify possible issues.

3. Defect Logging: Every identified defect is recorded in a tracking tool with a unique ID, clear title, description, reproduction steps, expected result, actual result, and supporting evidence.

4. Defect Classification: Defects are grouped by type, module, severity, and other relevant characteristics. Classification helps identify common problem areas and assign the correct specialists.

5. Defect Prioritization: Defects are ranked according to their urgency and business importance. This ensures that problems affecting critical functions, security, or major releases receive appropriate attention.

6. Defect Assignment: Each defect is assigned to a responsible developer or team. Clear ownership prevents defects from being overlooked and makes progress easier to monitor.

7. Defect Investigation: The assigned person reproduces the issue, examines logs or code, identifies the cause, and determines the required correction.

8. Defect Resolution: The developer fixes the problem or records an appropriate decision if it is invalid, duplicated, or deferred. The defect status is updated to reflect the action taken.

9. Retesting: The tester checks the corrected functionality to verify that the original problem has been resolved.

10. Regression Testing: Related and existing features are tested to ensure that the correction has not introduced new defects elsewhere in the application.

11. Defect Closure: The defect is closed after successful verification and satisfaction of the project's closure criteria. If the correction fails, the defect may be reopened.

12. Defect Reporting and Metrics Analysis: Reports are generated to summarize the number of defects, their current states, severity levels, resolution time, and recurring patterns. These metrics help management monitor quality and progress.

Q9. Explain the role of Defect Management Tools in software testing.

Defect Management Tools are software applications used to record, assign, track, prioritize, monitor, and report defects throughout the software development and testing process. They provide a centralized location for maintaining defect information and coordinating work between testers, developers, and project managers.

Examples of commonly used defect management tools include Jira, Bugzilla, Azure DevOps, and MantisBT.

Role of Defect Management Tools in software testing:

1. Centralized Defect Recording: These tools maintain a single repository of defect reports. Team members can access the same information instead of relying on separate spreadsheets, emails, or informal messages.

2. Defect Assignment and Tracking: Tools allow defects to be assigned to responsible developers and monitored through different states, such as New, Assigned, In Progress, Fixed, and Closed.

3. Severity and Priority Management: They support the classification of defects according to severity and priority, helping the team focus on the most important problems.

4. Improved Communication: Testers and developers can add comments, attach evidence, request clarification, and share updates directly within a defect record. This reduces misunderstandings.

5. Automated Notifications: Many tools can notify team members when a defect is assigned, updated, reopened, or resolved. Notifications help ensure that required actions are not missed.

6. Progress Monitoring: Dashboards display defect counts, open and closed issues, unresolved critical defects, and other useful information. Managers can monitor testing progress and identify bottlenecks.

7. Reporting and Metrics: Tools generate reports about defect trends, resolution time, severity distribution, and team workload. These reports support quality assessment and project planning.

8. Integration with Other Tools: Defect management tools can integrate with test management systems, source-code repositories, build pipelines, and continuous integration tools. This helps connect defects with test results and code changes.

9. Maintaining History and Traceability: They preserve defect updates, comments, assignments, and resolution details. This creates an audit trail and helps teams understand how an issue was handled.

10. Supporting Collaboration: Distributed teams can work on the same defect records, making it easier to coordinate testing and development across locations and departments.

Q10. What are the important features of a good Defect Tracking Tool? Explain.

A Defect Tracking Tool is an application used to record and monitor software defects from the time they are reported until they are resolved and closed. A good tool should be easy to use, reliable, flexible, and capable of supporting the complete defect management process.

Important features of a good Defect Tracking Tool:

1. Easy Defect Logging: The tool should allow testers to create defect reports quickly using clear forms containing fields for title, description, reproduction steps, expected result, and actual result.

2. Unique Defect Identification: Each defect should receive a unique ID. This makes it easy to search for, reference, discuss, and track individual issues.

3. Defect Status Management: The tool should support configurable workflows for states such as New, Assigned, In Progress, Fixed, Retest, Reopened, and Closed. This helps monitor each defect's progress.

4. Severity and Priority Assignment: It should allow defects to be classified according to technical impact and urgency. This helps the team decide which issues require immediate attention.

5. Assignment and Ownership: The tool should support assigning defects to specific team members or teams. It should also make the current owner and responsibility clear.

6. Search and Filtering: Users should be able to search defects by ID, title, status, severity, priority, assigned person, module, or date. Filtering makes it easier to locate important issues.

7. Notifications and Alerts: Automatic notifications should inform team members about new assignments, status changes, comments, and reopened defects.

8. Attachments and Evidence: The tool should support screenshots, logs, videos, and other files that help developers understand and reproduce defects.

9. Reporting and Dashboards: It should provide reports and dashboards showing open defects, closed defects, defect trends, severity distribution, resolution time, and testing progress.

10. Integration Capabilities: A good tool should integrate with test management software, source-code repositories, development environments, and continuous integration systems where needed.